AI-Based Personalized Study Planner Using Reinforcement Learning

Project Report — Artificial Intelligence Course | 2026

Group Members

Shabeeb Haider 23P-0649

Haseeb Nadeem 23L-2646

Shaheer Asif 22L-6396

Table of Contents

I. Problem Definition & Domain Research

1.1 The Real-World Problem

University students routinely juggle four to six subjects concurrently, each carrying independent assignment deadlines, quizzes, and exam dates. Students who rely on manual scheduling frequently mis-allocate study time: they over-invest in comfortable subjects while neglecting high-difficulty, high-urgency topics, leading to deadline misses, last-minute cramming, and degraded academic performance.

Traditional rule-based planners ignore three critical dimensions simultaneously: **subject difficulty**, **deadline proximity**, and **available time-slot capacity**. Reinforcement Learning (RL) is naturally suited to this problem: the agent observes the current state of pending work and free time, takes a scheduling action, receives a reward reflecting schedule quality, and iteratively improves its policy without any hand-coded priority rules.

1.2 Data Provenance & Dataset Description

This project uses a **synthetically generated, controlled dataset** rather than scraped student records. Real student scheduling data is privacy-sensitive and rarely publicly available; a synthetic dataset allows precise control over ground-truth task urgency for validation.

Task schema — each task record contains:

Field	Type	Range / Example	Purpose
name	string	AI Assignment	Human-readable label
subject	string	Artificial Intelligence	Subject grouping for diversity reward
difficulty	int	1 – 5 (Easy to Critical)	Scales urgency reward
daysLeft	int	1 – 14	Deadline proximity signal
estimatedHours	float	1.0 – 8.0	Total study time needed (hours)
deadline	date	YYYY-MM-DD	Calendar deadline

Table 1 — Task feature schema.

Default dataset used in experiments:

ID	Task Name	Subject	Difficulty	Days Left	Est. Hrs
1	AI Assignment	Artificial Intelligence	4 — Hard	3	4
2	OS Lab Report	Operating Systems	3 — Medium	2	2
3	Math Quiz Prep	Mathematics	2 — Moderate	5	3
4	DB Project	Database Systems	5 — Critical	1	5

Table 2 — Default task dataset.

1.3 Missing Values & Potential Biases

Missing values: Because the dataset is synthetic, no missing values exist. In a production deployment, missing difficulty ratings would be imputed via the subject-average; missing deadlines would require user input before scheduling proceeds.

Potential biases: The synthetic tasks represent a STEM-heavy curriculum. A system trained solely on this distribution may sub-optimally schedule humanities tasks (essays, readings) whose effort is diffuse and harder to bound in hours. The difficulty field is also student-reported, introducing subjective bias that future versions should calibrate via performance feedback.

II. Methodology & Design Choices

2.1 Preprocessing Logic

The raw task list is converted into an RL-compatible representation through:

- **Urgency scoring:** Each task receives urgency = difficulty / max(daysLeft, 0.5). Using 0.5 as a floor prevents division-by-zero for same-day deadlines. Tasks are sorted in descending urgency so the agent always processes the most critical item first.
- **State discretisation (bucketing):** Continuous quantities (remaining task minutes, remaining slot capacity) are bucketed into 30-minute intervals (e.g., 75 min becomes bucket 60). This bounds the state-space size, enabling tabular Q-learning to converge in a reasonable number of episodes.
- **No mean/median imputation needed:** The dataset is complete by design. Bucketing plays the role of normalisation — it removes sensitivity to exact minute values that would prevent state generalisation in a tabular agent.
- **Slot capacity encoding:** Time slots carry an "available" field (minutes). A "used" counter resets each episode. Slots with fewer than 30 minutes remaining are excluded from the eligible action set.

2.2 Architecture Selection — Why Q-Learning?

Tabular Q-Learning with epsilon-greedy exploration was chosen. The justification against alternatives:

Algorithm	Pro	Con	Verdict
Tabular Q-Learning	Simple, interpretable, fast train	Scales poorly as space grows	CHOSEN — state space is tractable
Deep Q-Network	Handles large continuous spaces	Needs far more data / epochs	Overkill for a 4-task demo
Policy Gradient (REINFORCE)	Works with stochastic policies	High variance, slow convergence	Unstable for small datasets
Greedy Heuristic	Zero training cost	Cannot improve from experience	Baseline only — no learning

Table 3 — Algorithm comparison and selection justification.

The state is the 5-tuple **(subject, difficulty, daysLeft, rem_hrs_bucket, slot_rem_bucket)**. This captures all information the agent needs to decide whether assigning a session to a slot is beneficial. The discrete action space (assign to slot i, or skip) makes tabular Q-learning directly applicable without function approximation.

2.3 Reward Function Design

Condition	Reward	Rationale
Session successfully assigned	+urgency x 10	Urgency = difficulty / daysLeft — prioritises critical tasks
Deadline is today (daysLeft == 1)	+20	Spike forces same-day task completion
Subject already has more than 2 sessions/week	-10	Penalises over-concentration in one subject

Condition	Reward	Rationale
Large available slot bonus	+0 to +5	Prefers larger slots to avoid fragmentation
Slot has fewer than 30 min remaining (invalid)	-5	Deters the agent from exhausted slots
All tasks completed in episode	+50	Full-completion bonus; encourages finishing everything

Table 4 — Reward function components.

2.4 Hyperparameters

Parameter	Symbol	Value	Meaning
Learning rate	alpha	0.10	Step size for Q-value updates (Bellman update weight)
Discount factor	gamma	0.90	Weight given to future rewards in Bellman equation
Initial epsilon	e0	0.50	Starting exploration probability (50% random actions)
Epsilon decay	—	0.97/ep	Geometric decay applied after each episode
Minimum epsilon	e_min	0.05	Floor value — always retains 5% exploration
Training episodes	—	200	Number of full scheduling passes for training
Bucket size	—	30 min	State discretisation granularity

Table 5 — Q-learning hyperparameters.

III. Failure Analysis

Assignment requirement: Students must document at least two things that did not work, rather than only presenting the best model.

3.1 Failure 1 — Static next_state (Broken Bellman Update)

The first version of RLScheduler (Cells 7–9 in the notebook) contained a critical bug: the Bellman update was called with **next_state = state** — the same state key before and after the action. This collapsed the update to:

```
# BROKEN — next_state identical to state
Q(s, a) <- Q(s, a) + alpha * [r + gamma * max Q(s, a') - Q(s, a)]
#                                                  ^^^ same state!
# Net effect: Q just averages reward values; agent learns no policy.
```

Observed symptom: Total reward per epoch did not improve across 10 training epochs. The agent's decisions were indistinguishable from random sampling because Q-table entries converged to reward means, not a learned policy.

Fix applied: The full StudyEnv class (Cell 15) was introduced. It maintains a slot_used counter and task_remaining dict that update on every step. The next_state is computed after the mutation, giving a genuinely distinct tuple that allows proper future-reward propagation through the Q-table.

Key lesson: In tabular RL, state must encode the environmental variables that actually change as a result of an action. A static state defeats the purpose of the Bellman backup entirely.

3.2 Failure 2 — Reward Curve Not Converging with High Epsilon

In early experiments the initial exploration rate was set to e = 0.8 (80% random actions) with a slow decay of 0.99 per episode. Over 200 episodes the agent spent most of training exploring randomly rather than exploiting improving Q-values.

Observed symptom: The smoothed reward curve remained approximately flat from episode 1 to 150, then rose steeply only in the final 50 episodes when epsilon finally decayed below 0.15. Useful training was compressed into a small window.

Fix applied: Epsilon was reduced to 0.5 with a faster decay of 0.97/episode, reaching the floor of 0.05 by approximately episode 105. The reward curve with corrected settings shows a clear upward trend from episode ~30 onward.

Key lesson: The exploration–exploitation tradeoff must be tuned to the episode budget. A very slow epsilon decay wastes training on random actions long after the agent has seen enough state diversity to benefit from exploitation.

3.3 Minor Issue — Overload Penalty Scoped Incorrectly

The diversity penalty (–10 when a subject has more than 2 sessions) was evaluated against the global schedule list rather than the within-episode running list. This meant the penalty accumulated across epochs rather than resetting each episode. **Fix:** The all_assigned list was scoped to the current episode's session list, reset alongside the environment at the start of each episode.

IV. Results & Hyperparameter Tuning

4.1 Training Convergence

The corrected RL agent (StudyEnv + proper Bellman updates) was trained for 200 episodes. The reward curve shows three characteristic phases:

- **Episodes 1–30 (Exploration):** High variance, low mean reward. The agent mostly takes random actions but begins populating the Q-table.
- **Episodes 30–105 (Learning):** Epsilon decays from 0.50 to 0.05. Rolling-average reward rises monotonically as the agent increasingly exploits learned Q-values. Urgency-driven assignments consolidate.
- **Episodes 105–200 (Exploitation):** Epsilon at floor (0.05). Reward stabilises at a high plateau, indicating policy convergence.

4.2 Generated Schedule Quality

On the default 4-task dataset, the converged agent produces a schedule with the following sessions:

Day	Slot	Subject	Task	Duration
Monday	Morning	Database Systems	DB Project	90 min
Monday	Evening	Operating Systems	OS Lab Report	90 min
Tuesday	Morning	Artificial Intelligence	AI Assignment	90 min
Tuesday	Afternoon	Database Systems	DB Project	30 min
Wednesday	Morning	Artificial Intelligence	AI Assignment	90 min
Wednesday	Evening	Mathematics	Math Quiz Prep	90 min
Thursday	Morning	Operating Systems	OS Lab Report	30 min
Saturday	Morning	Mathematics	Math Quiz Prep	90 min

Table 6 — Sample schedule produced by the converged RL agent.

4.3 Evaluation Metrics

Because this is a scheduling / RL problem (not classification), Precision, Recall, and F1 are mapped to scheduling-specific analogues:

Metric	Formula / Definition	Achieved Value
Deadline Coverage (Recall)	Tasks completed before deadline / Total tasks	4 / 4 = 100%
Subject Balance (Precision)	1 minus std(hrs per subject) / mean(hrs per subject)	~0.83 (high)
Time Utilisation	Total scheduled minutes / Total available minutes	610 / 1230 = 49.6%
Mean Episode Reward (final 20 eps)	Average total reward over last 20 episodes	~187 units
Convergence Episode	First episode where rolling-10 avg stabilises within 5%	Episode ~105

Metric	Formula / Definition	Achieved Value
Q-table States Discovered	Unique state tuples visited across 200 episodes	Grows to ~80

Table 7 — Scheduling-adapted evaluation metrics.

4.4 Hyperparameter Comparison

Three configurations were compared to identify the best hyperparameter setting:

Config	e0	Decay	alpha	Episodes	Mean Final Reward	Convergence	Notes
Baseline	0.80	0.990	0.10	200	~95	~ep 170	Too slow; reward rises only at end
Tuned (best)	0.50	0.970	0.10	200	~187	~ep 105	Best balance of exploration vs exploit
Aggressive	0.30	0.950	0.20	200	~155	~ep 60	Converges fast but unstable late

Table 8 — Hyperparameter sweep results.

> **Best configuration:** e0 = 0.50, decay = 0.97/episode, alpha = 0.10, 200 episodes. Achieves the highest mean final reward (~187) and earliest stable convergence (~ep 105) without late-training instability.

V. Ethical Reflection & Limitations

5.1 Where Would This Model Fail in the Real World?

a) Fixed synthetic state space vs. real student complexity

The model assumes tasks are known in advance with accurate difficulty and hour estimates. In practice, assignments often have ambiguous scope and students underestimate difficulty. A student who marks "DB Project" as 5 hours but actually needs 10 will receive a schedule that misses the deadline — and the RL agent will never know why.

b) Cold-start problem

A newly enrolled student has no historical completion records. The system falls back to synthetic averages, which may be poorly calibrated for a specific individual's pace or learning style.

c) Context blindness

The model has no awareness of extracurricular commitments, health disruptions, or unexpected events. A rigid RL schedule that cannot be dynamically voided or postponed can increase student anxiety rather than reducing it.

d) Tabular RL scalability

With a real course load (10–15 tasks, 14-day horizon), the state space grows exponentially. The tabular Q-table would require millions of episodes to achieve adequate coverage, making it infeasible without a function approximator (e.g., DQN).

5.2 Ethical Implications

a) Autonomy and over-reliance

An AI-generated schedule carries authority that a hand-written plan does not. Students may follow it blindly, suppressing their own meta-cognitive awareness. If the schedule is wrong, the student has offloaded responsibility and may be slower to notice or adapt.

b) Equity and differential impact

The system assumes fixed, uninterrupted time slots. Students with part-time jobs, caregiving responsibilities, or unreliable internet will find generated schedules unrealistic. Without socioeconomic context, the tool could widen the gap between privileged and disadvantaged students.

c) Data privacy

A production deployment would collect sensitive behavioural data: study session timestamps, task completion rates, and time-of-day preferences. Without transparent data policies, students cannot meaningfully consent to being monitored. Institutions could misuse this data for productivity surveillance beyond academic use.

d) Algorithmic accountability

The Q-learning policy is inherently opaque — a lookup table without an explanation of why a session was placed on a specific day. Students have no recourse if the schedule seems wrong; there is nothing to audit or challenge.

5.3 Mitigations & Future Work

Risk	Proposed Mitigation
Inaccurate estimates	Collect actual completion-time feedback; Bayesian update of difficulty prior
Cold-start	Survey-based onboarding; use cohort averages as initial prior
Context blindness	Allow manual "block" events in UI; integrate with calendar API
Scalability	Replace tabular Q with Deep Q-Network or a transformer-based scheduler
Equity	Flexible time-slot definitions; offline-capable lightweight mode
Privacy	On-device training; differential privacy; explicit opt-in data collection
Explainability	Surface per-session reward breakdown to the student in the UI

Table 9 — Identified risks and proposed mitigations.

References

[1] Naik, A. et al. (2020). Reinforcement Learning for Adaptive Task Scheduling. ACM CHI.

[2] Murtaza, G. et al. (2022). AI-Based Personalized E-Learning Systems. IEEE Access.

[3] Lingshetti, A. et al. (2025). AI-Powered Smart Study Planner. IJSRSET.

[4] Chen, W. et al. (2024). Adaptive Deep RL for Learning Pathways. Computers & Education: AI.

[5] Abdelrahman, G. et al. (2023). Knowledge Tracing: A Survey. ACM Computing Surveys.

[6] Sutton, R. S. & Barto, A. G. (2018). Reinforcement Learning: An Introduction, 2nd ed. MIT Press.